

Porting `igsca_tree` / `igsca_ctree` into `doTrees()`

What is missing, what is wrong, and what to change — branch `igscaTrees`

1 Scope and method

This reviews the port of `dev/igsca/tree_template/igsca_tree.R` and `igsca_testers.R` into `R/postestimate_doTrees.R` + `R/helper_doTrees.R`, and the new `tests/testthat/test-postestimate_doTrees.R`.

Every finding below was **executed**, not read off. All five **influence** values were run against `tests/testthat/data/igscaTrees.Rdata` ($n = 1000$, covariates `z_true` (2-level factor), `noise_1`, `noise_2`) under `devtools::load_all()`, with reduced `R_test/maxdepth/max_cuts` where the default control was too slow to finish. Verbatim error text is quoted.

Revision history. The first pass found the partition family unrunnable. Since then:

- **Second pass** — `idx_permutation()` and `ndt_dists()` were added (`R/helper_doTrees.R`:928, :959) and `devtools::document()` was run. Verified in B1 and P1.
- **Third pass** — `splitter` became a `match.arg()` string and the test file was updated to match. Verified in B2 across all 17 configurations the tests exercise, plus both guard paths.
- **Fourth pass** — the influence function moved to its own `influence_fn` local (B2), and S1 was fixed: `argmax_split()` now counts failed kernel scans, `doTrees()` warns on a wholly dead kernel, and the three tests of section 5.4 are implemented and passing.
- **Fifth pass** — S2's dropped `cc$bonferroni` line was restored and verified on both distributions. Section 3.2.1 now answers the `testtype` TODO properly, including why the asymptotic route is genuine COIN inference and why partykit's "Bonferroni" is really Šidák.
- **Sixth pass** — S3 fixed: leaves are refit after growing and attached to the terminal nodes, so `coef()` and `plot()` work and the tree finally carries a per-leaf IGSCA fit. Inner-node objects are kept, with an opt-in switch.

All behavioural findings are now closed. Every `influence` $\times$ `splitter` combination runs, and the three silent failures are fixed. What remains is a test suite that still cannot detect regressions in most of it (T1–T3, T5) and packaging cleanup (P1–P6). Section 5 sets out how to test the mixed-splitter configurations; its injection recipes have been rewritten for the string-valued API, and 5.4 is now shipped code rather than a proposal.

1.1 Status at a glance

ID	Finding	Status
B1	<code>idx_permutation()</code> / <code>ndt_dists()</code> not ported	resolved
B2	<code>splitter</code> is a function in one branch, a string in the other	resolved
S1	Broken split kernel degrades to a stump with no diagnostic	resolved
S2	<code>control\$bonferroni</code> silently ignored on the <code>ctree</code> path	resolved
S3	<code>coef()</code> / <code>plot()</code> empty for any tree that splits	resolved
T1	Snapshots stale (<code>trees_out</code> vs <code>trees_mx</code>)	open
T2	<code>length(tree) == 5</code> is a disguised structural assertion	open
T3	<code>expect_no_error()</code> cannot see the real failure mode	open
T4	Default control makes the partition tests unrunnable	changed
T5	Collector diagnostics and the scan cache are untested	open
P1	NAMESPACE stale	changed

ID	Finding	Status
P2	<code>cSEM:::</code> / <code>cSEM::</code> into the package's own namespace	open
P3	Roxygen placeholders; wrong <code>@return</code>	open
P4	Argument naming departs from package convention	open
P5	<code>fit_csem()</code> hard-codes the estimator	open
P6	No input validation	open

The test file has been updated to pass strings and has gained section 5.4's three blocks — but its original five blocks are untouched, so T1–T3 stand and T4 is only half-addressed: `ctl_mixed` now exists at the foot of the file while all 17 original calls still use a bare `igsca_tree_control()`. T4 is what will actually stop a run completing.

2 Resolved

2.1 B1. `idx_permutation()` and `ndt_dists()` are now available

Location: R/helper_doTrees.R:928, R/helper_doTrees.R:959

Both helpers now exist, and both satisfy the contracts their call sites require. This was the sole blocker on the partition family; it is cleared.

`idx_permutation()` — orientation is correct. The returned matrix is $R \times n$ (`nrow = R`, `ncol = n`), which is what `permutation_pvalue()` needs when it indexes `goes_left[perm[r, 1]]`. Had it come back transposed, the result would have been a recycled-length nonsense null distribution rather than an error, so this is worth having confirmed explicitly. The sample-without-replacement-within-stratum construction matches `boot::permutation.array` as the docstring claims.

`ndt_dists()` — names and argument order match the call site. It returns `c(DGc, DGi, DLc, DLi)`, and `partition_stat()` reads `unname(ds[stat_kind])` for `stat_kind` in "DGi" / "DLi". The `(Sc_pool, Si_pool, mga_fit)` signature lines up with `ndt_dists(coll$ndt_pools$Sc, coll$ndt_pools$Si, mga$fit)` at R/helper_doTrees.R:635.

Verified end to end. The partition family, previously unrunnable, now grows trees. At `R_test = 24L`, `max_cuts = 3L`, `maxdepth = 1L`, `minbucket = 200L` (14.8 s):

```
Fitted party:
[1] root
|   [2] z_true in 1: *
|   [3] z_true in 2: *

      z_true      noise_1      noise_2
statistic 0.04315312 0.001707126 0.001593087
p.value   0.04000000 0.280000000 0.083333333
criterion -0.04082199 -0.328504067 -0.087011377
```

The DLi/DGi kernels agree exactly with the reference implementations used to pre-verify the path in the first pass (`argmax` cutpoint on `noise_1 = -1.69654` for both, `-0.231765` for FIT), so the shipped `ndt_dists()` computes what that analysis assumed.

Two loose ends, both minor:

- **The `strata` argument is dead.** `idx_permutation(n, R, strata = NULL)` implements stratified permutation, but `permutation_pvalue()` (R/helper_doTrees.R:699) calls `idx_permutation(n, R)` with no `strata`. If stratified permutation within the node was intended — which would be the natural thing to want when the node is already conditioned on a covariate — it is not wired up. If it was not intended, drop the parameter rather than leave an untested branch in the package.
- **The provenance comment is now wrong.** R/helper_doTrees.R:535 still says these come from R/MGA/csem_test_helpers.R. They are defined 400 lines below it. (`dev/igsca/tree_template/csem_test_helpers.R` is now present but untracked.)

2.2 B2. `splitter` is now a single `match.arg()` string

Location: R/postestimate_doTrees.R:16--29

Implemented as recommended — one string-valued argument with an explicit "native" level, resolved once before either branch:

```
doTrees <- function(data, model, covariates,
                    influence = c("mat", "vec", "FIT", "DLi", "DGi"),
                    splitter   = c("native", "FIT", "DLi", "DGi"),
```

```

        control    = igsca_tree_control() {
influence <- match.arg(influence)
splitter  <- match.arg(splitter)

split_fn <- switch(splitter,
  native = NULL, FIT = split_max_fitdiff,
  DLi = split_max_dli, DGi = split_max_dgi)

```

`split_fn` is consumed by the `ctree` branch at :117 and by `grow_extree()` at :221. Both original failure modes are gone, and the structural hazard behind the second one is closed: `match.arg()` guarantees a length-1 character, so `splitter %in% c(...)` can no longer evaluate to `logical(0)` and slip through `stopifnot()` vacuously.

The retained `stopifnot()` at :167 is **not** redundant — it now does real work, rejecting "native" on the partition path, where there is no engine-native scan and a `NULL` kernel would die inside `argmax_split()`'s `tryCatch` and silently yield a stump. Both guards produce readable errors:

```

doTrees(dat, model, covs, influence = "FIT", splitter = "native")
#> Error: splitter should be any one of 'FIT', 'DLi' or 'DGi' when the influence
#>       (selector) function is one of 'FIT', 'DLi' or 'DGi'

doTrees(dat, model, covs, influence = "mat", splitter = "nope")
#> Error: 'arg' should be one of "native", "FIT", "DLi", "DGi"

```

Verified sweep. All 17 configurations the test file exercises now run, none error, and every one produces a split:

Configuration	width	n_fail (node/res)	time
mat × native	3	0 / 0	0.9 s
mat × {FIT, DLi, DGi}	3	0 / 2	1.7–2.0 s
vec × native	3	0 / 0	0.3 s
vec × {FIT, DLi, DGi}	3	0 / 0	1.5–1.8 s
{FIT, DLi, DGi} × {FIT, DLi, DGi}	2	0 / 0	14.0–15.3 s

(`ctree` rows at `R_test = 50`, `maxdepth = 2`, `max_cuts = 8`; partition rows at `R_test = 24`, `max_cuts = 3`, `maxdepth = 1`, `minbucket = 200`. The `n_fail_resample = 2` on the `mat` mixed rows is ordinary non-convergence at extreme cutpoints, not a fault.)

Residual nits: both since fixed. The obsolete “add a switch statement” TODO has been deleted, and the mid-function rebinding of `influence` is gone — :45 now assigns the influence function to its own local, `influence_fn`, read at :79:

```

influence_fn <- switch(influence, mat = influence_mat, vec = influence_vec)
...
h <- influence_fn(E)

```

`influence` therefore stays a character for the whole call, so the `else if` (`influence %in% c("FIT", "DLi", "DGi")`) at :155 now tests what it appears to test. (An intermediate revision named this local `selector`, which collided with the genuine `extree` `selectfun` of the same name at :158/:219 — two different roles under one identifier in one function body. `influence_fn` avoids that: `selector` is now unambiguously the variable-selection function `partykit` means by the term.)

Re-running the full sweep after the rename gave results **byte-identical** to the pre-rename run — every `width` and `n_fail` value unchanged, differing only in wall-clock jitter. As an independent check that the `switch()` still maps to the right function, `influence = "mat"` and `"vec"` produce different trees, and their node-2 cutpoints (`-1.09783` on `noise_1`, `1.46613` on `noise_2`) reproduce the recorded snapshot exactly. That is a real cross-check rather than a coincidence: on the native path the cutpoint comes from `ctree`'s deterministic `maxstat` scan, so it is invariant to `R_test` and the seed.

3 Silent failures — all three now resolved

3.1 S1. A broken split kernel degraded to a stump with no diagnostic

Location: R/helper_doTrees.R:293--345 (argmax_split, warn_dead_splitter)

Status: fixed. `argmax_split()` wraps each kernel call in `tryCatch(..., error = function(e) NA_real_)`, which is right for a genuinely non-identified candidate partition but also swallowed **programming** errors: every candidate became NA, `any(is.finite(stats))` was FALSE, the loop nexted through every covariate, `argmax_split()` returned NULL, and partykit read that as “no admissible split”. A dead kernel and an unsplittable node were indistinguishable in the returned object — which is why the original `ndt_dists()` defect had to be found by hand rather than by the suite.

The `tryCatch` is unchanged (it still has to absorb non-convergent candidates). What changed is that the two cases are now counted apart. `argmax_split()` tracks whether the kernel was **evaluated at all** on this invocation, and only a scan that ran and yielded nothing is recorded as a failure:

```
stats <- vapply(cands, function(cc) {
  tryCatch(splitter(model, mf, subset, cc$goes_left, ctrl),
    error = function(e) NA_real_)
}, numeric(1))
## The kernel was actually evaluated on this covariate, so the invocation
## counts as a scan regardless of what came back.
scanned <- TRUE
if (!any(is.finite(stats))) next
collector$n_split_scan <- collector$n_split_scan + 1L
return(cands[[which.max(stats)]]$split)
}
if (scanned) {
  collector$n_split_scan <- collector$n_split_scan + 1L
  collector$n_fail_split <- collector$n_fail_split + 1L
}
NULL
```

The distinction is load-bearing: a node with no admissible partition exits via `if (!length(cands)) next` and never sets `scanned`, so it is **not** counted as a kernel failure. Both counters were added to `new_collector()` and both are surfaced on the returned tree:

```
attr(tr, "igsca_info")$n_split_scan # scans where the kernel actually ran
attr(tr, "igsca_info")$n_fail_split # ... of which produced nothing usable
```

A totally dead kernel — `n_fail_split == n_split_scan` — now also warns, via a new internal `warn_dead_splitter()` called from both branches of `doTrees()` just before the attribute is attached:

```
Warning: The 'DLi' split kernel produced no usable statistic at any of the 2
node(s) where it was scanned, so no split point could be chosen from it and the
tree was grown as if no split were admissible. Check that the kernel runs
standalone via partition_stat().
```

The warning deliberately fires only on total failure. A **partial** failure (some nodes fine, others not) is ordinary non-convergence and would be noise as a warning — it is visible in the counters instead, where it can be asserted on. `splitter = "native"` short-circuits the check and leaves both counters at 0, since no kernel is involved.

Tests are in section 5.4 and now pass.

3.2 S2. control\$bonferroni was silently ignored on the ctree path

Location: R/postestimate_doTrees.R:116

Status: fixed. The dropped line has been restored:

```
cc <- partykit::ctree_control(...)
cc$bonferroni <- isTRUE(control$bonferroni)
```

It was never a no-op. `ctree_control()` derives `bonferroni` from `testtype`, but the port computes `testtype` from `coin_distribution` first, so with the default `coin_distribution = "approximate"` it always sent `testtype = "MonteCarlo"` and `cc$bonferroni` was always `FALSE` — while on the **partition** path `grow_extree()` copies the whole control list into `ctrl`, so the same argument did take effect there. Verified fixed on both distributions (`R_test = 200`, three covariates):

		z_true	noise_1	noise_2
asymptotic	FALSE	5.31e-57	0.872895	0.810172
asymptotic	TRUE	1.59e-56	0.997947	0.993160
MonteCarlo	FALSE	0.000	0.890	0.845
MonteCarlo	TRUE	0.000	0.998669	0.996276

3.2.1 Answering the TODO: testtype vs bonferroni

The earlier draft of this section said `testtype` picks “permutation vs asymptotic”. That is half the story and worth stating properly, because `testtype` conflates **two** dimensions:

testtype	p-value from	Multiplicity
"Bonferroni"	asymptotic limiting distribution	adjusted
"Univariate"	asymptotic limiting distribution	none
"MonteCarlo"	nresample permutations	none
"Teststatistic"	no p-value — raw statistic is the criterion	n/a

Are the p-values not always permutation-based? No — and this is the crux. In the conditional-inference (COIN) framework “permutation test” describes the **null distribution**, not the **computation**. Strasser and Weber derived the conditional mean μ and covariance Σ of the linear statistic T under that permutation null **in closed form**. Standardising with them gives $c = (T - \mu)^\top \Sigma^+ (T - \mu)$, which is asymptotically χ^2 with $\text{rank}(\Sigma)$ degrees of freedom (`teststat = "quadratic"`). You can then read the p-value off that χ^2 limit **without ever drawing a permutation**. That is the asymptotic route, and it is partykit’s default.

Two measurements confirm no resampling happens on that path — same seed, `influence = "mat"`, root criteria:

approximate, R_test = 50 :	z_true 0.00	noise_1 0.74	noise_2 0.82
approximate, R_test = 200:	z_true 0.000	noise_1 0.890	noise_2 0.845
asymptotic, R_test = 50 :	z_true 5.31e-57	noise_1 0.872895	noise_2 0.810172
asymptotic, R_test = 200:	z_true 5.31e-57	noise_1 0.872895	noise_2 0.810172

First, 5.31e-57 is unreachable for **any** Monte-Carlo test at `R = 50` or `200`. Second, the asymptotic values are bit-identical across the two `R_test` settings — `nresample` is simply unused. So `coin_distribution = "asymptotic"` is a real setting with the same conditional null and the same μ , Σ ; it only swaps an empirical tail for a χ^2 tail. It is far cheaper, and it yields continuous p-values instead of a grid; Monte Carlo is the safer choice in small nodes where the asymptotics have not bitten.

Why the fix is still needed. `.extree_node()` applies the adjustment to whatever p-values it was handed, independently of how they were produced:

```
if (ctrl$bonferroni)
  sf$criteria["p.value", ] <- sf$criteria["p.value", ] *
    sum(!is.na(sf$criteria["p.value", ]))
```

So the two knobs **are** orthogonal at the point of use, even though `testtype` bundles them at the point of configuration — which is exactly why an explicit assignment was required.

Two details worth knowing. That row holds $\log(1 - p)$, not p , so multiplying by k gives $\log((1 - p)^k)$ — the adjustment is Šidák, $1 - (1 - p)^k$, not Bonferroni's kp . It agrees with Bonferroni to first order (hence the exact $\times 3$ on `z_true` above) but not for large p : $1 - (1 - 0.872895)^3 = 0.997947$, matching the table exactly. Second, `partykit` **does** support asking for both dimensions at once — `ctree_control()` accepts a length-2 `testtype`:

```
testtype <- match.arg(testtype, several.ok = TRUE)
if (length(testtype) > 1) {
  stopifnot(all(testtype %in% c("Bonferroni", "MonteCarlo")))
  ttesttype <- "MonteCarlo"
}
... bonferroni = "Bonferroni" %in% testtype
```

`ctree_control(testtype = c("Bonferroni", "MonteCarlo"))` is identical to the manual override on every non-function component (verified; the function-valued slots differ only by closure environment). So the mapping could drop the override entirely and stay inside the documented API:

```
testtype <- c(
  if (isTRUE(control$bonferroni)) "Bonferroni",
  if (control$coin_distribution == "approximate") "MonteCarlo"
)
if (is.null(testtype)) testtype <- "Univariate"
```

which reaches all four combinations — MC $\pm$ adjustment, asymptotic $\pm$ adjustment — with no post-hoc surgery on the control object. Optional: the current code is correct as it stands.

3.3 S3. `coef()` and `plot()` were empty for any tree that splits

Location: `R/helper_doTrees.R` (`attach_leaf_fits`, `drop_inner_node_objects`, `coef.igsca_tree`, `plot.igsca_tree`); wired in at `R/postestimate_doTrees.R`:150--160 and :230--240

Status: fixed. Both methods read `nobs / objfun` out of `info_node()` at **terminal** nodes, but `partykit` only calls the trafo at nodes it **attempts to split**, so leaves arrived with `info = NULL`. `coef()` returned `NULL` for every tree with at least one split (it only appeared to work on stumps, where the root **is** the leaf) and `plot()` drew unlabelled boxes, because `sprintf("n = %s", NULL)` yields `character(0)` rather than erroring.

The deeper problem was not the methods: **the tree carried no per-leaf IGSCA fit at all**. A new `attach_leaf_fits()` refits the model once per leaf after growing and writes the result into the terminal nodes' own `info`, so both methods now work through the ordinary `info_node()` route rather than a side table.

Two properties were verified before choosing that design:

- `as.list()` / `as.partyndode()` round-trips a grown tree **identically**, so rewriting only the leaf entries leaves the rest of the structure untouched.

- Terminal-node info is invisible to `print()` — partykit’s `formatinfo_node()` falls back to `"*"` for list-valued info — and that holds even when the info carries a full `cSEMResults`. Attaching leaf fits therefore does not change printed output or disturb the snapshots.

Measured, `maxdepth = 2` on the `ctree` path and a `FIT × FIT` partition tree:

<code>ctree</code>	<code>coef()</code>	<code>nobs</code>	<code>objfun</code>	<code>partition</code>	<code>coef()</code>	<code>nobs</code>	<code>objfun</code>
	node 3	68	NA		node 2	500	3553.173
	node 4	432	3076.803		node 3	500	3438.003
	node 5	500	3438.003				

Both previously returned `NULL`. `coef()` also gained row names (node ids) and now emits an `NA` row instead of silently dropping a node — `rbind()` discards `NULL`s, which is precisely how the method used to collapse a whole tree to `NULL`. `plot()`’s default panel prints `?` rather than an empty box when a value is missing. `objfun` is the sum of squared casewise GSCA residuals, matching what the `ctree` trafo stores on inner nodes; on the partition path it is the **only** source of an SSR, since that trafo sets `objfun = NA_real_` throughout.

What the fix exposes. Leaf refits can fail, and now say so: a new `n_fail_leaf` counter joins the `igsca_info` attribute, and the failing leaf gets `converged = FALSE`, `objfun = NA`, `object = NULL`. The `maxdepth = 2` tree above hit exactly that at node 3 ($n = 68$). This is worth knowing when choosing `minbucket`: with 13 indicators the default 30L admits leaves the model may not fit. The failure is **sample-dependent rather than a size threshold** — random subsets of $n = 30$ and $n = 68$ fitted while $n = 50$ did not — so it is a rising probability of inadmissible estimates in small leaves, not a hard floor. Raising `minbucket` for larger models is the cheap defence; `n_fail_leaf > 0` is the signal that it is needed.

Inner-node objects: kept, with an opt-in switch. Every inner node’s info still holds a full `cSEMResults`, because `saveinfo = TRUE` persists whatever the trafo returned. `drop_inner_node_objects()` removes those payloads after growing, leaving criteria, `nobs`, `objfun` and all leaf fits intact. It is **not** called by default; a commented-out line in each branch of `doTrees()` marks the switch:

```
## MEMORY OPTIMIZATION (opt-in): each inner node's info holds a full
## cSEMResults object, so a maxdepth = 3 tree keeps up to 7 of them.
## Uncomment the next line to drop them; leaf fits are unaffected.
##   ret <- drop_inner_node_objects(ret)
## Do NOT instead delete `object = ft$fit` from the ytrafo above: a
## mixed-pair splitter reads model$object while the tree is still growing,
## so removing it there silently disables every non-native splitter.
```

That last warning is the point of putting the switch here rather than at the trafo. Deleting `object = ft$fit` looks like the obvious optimisation and is the one place it must not be done: on the `ctree` path `partition_stat()` reads `model$object` for every candidate partition of a mixed-pair splitter, and on the partition path the selectors read it too — removing it there would take out every non-native configuration, silently, exactly the way S1 used to fail. Measured saving from the safe switch: $1.9 \rightarrow 1.1$ Mb (`ctree`, 2 inner nodes) and $1.2 \rightarrow 0.7$ Mb (`partition`, 1 inner node), with `coef()` unchanged afterwards.

4 Test-suite findings

The test file is the right shape — one file per R file, five blocks covering the five methods, matching the conventions in `MEMORY.md` — and it has been updated to the new string-valued `splitter`, so it no longer errors on the signature. These are the things that still stop it from testing anything.

4.1 T1. Snapshots are stale and will fail

`tests/testthat/_snaps/postestimate_doTrees.md` still records the printed object under the name `trees_out` (lines 4 and 24):

```
# IGSCA Trees Conditional Test on Matrix of Residuals Runs as expected
```

```
Code
trees_out
```

The tests bind `trees_mx` / `trees_vec` / `trees_NPT_FIT` / `expect_snapshot()` captures the **expression** as well as the output, so both existing blocks fail on the `Code` line alone, and the three partition blocks have no snapshot at all. Delete the file and regenerate — the signature has settled, so this is now safe to do.

4.2 T2. `length(tree) == 5` is a disguised structural assertion

Location: `tests/testthat/test-postestimate_doTrees.R:32, :78, :125, :161, :196`

```
# Dirty substitute for snapshot
expect_true(length(trees_mx) == 5)
```

`length()` on a `party` dispatches to `length.party()`, which returns the **number of nodes**, not the length of a list. Measured: the same call with `maxdepth = 1L` gives `length(r) == 3`. So this asserts “the tree has exactly 5 nodes” — a strictly weaker duplicate of the snapshot immediately below it, which will break on any legitimate change to the control defaults or the RNG stream, for reasons the comment actively misleads you about.

It also interacts badly with T4: the reduced controls needed to make the suite finish produce 2- and 3-node trees, so all five assertions fail for a reason that has nothing to do with correctness. Drop them, or replace with assertions that say what they mean:

```
expect_s3_class(trees_mx, "igsca_tree")
expect_identical(attr(trees_mx, "igsca_info")$n_fail_full, 0L)
```

4.3 T3. `expect_no_error()` cannot see the failure mode that actually occurs

Location: `tests/testthat/test-postestimate_doTrees.R:35--62` and the four analogous blocks

All five blocks still wrap two or three `doTrees()` calls in a single `expect_no_error({...})`. Two problems. First, on failure you learn that **one** of three calls errored, not which. Second and more seriously, per S1 the realistic failure is not an error at all — a broken kernel returns a stump and the block passes. That is exactly what happened in the first pass: the DLi and DGi mixed splitters were entirely non-functional and the suite reported green.

This is now **more** misleading than before, not less. Every mixed configuration currently returns `width = 3`, so the blocks pass — and they would pass identically if all three kernels were dead. S1’s fix means such a tree now also carries `n_fail_split > 0` and emits a warning, but `expect_no_error()` reads neither: a warning is not an error, and nothing inspects the counters.

Split them, and assert on the artefact rather than on the absence of an exception:

```

for (sp in c("FIT", "DLi", "DGi")) {
  tr <- doTrees(dat, model, covs, influence = "mat", splitter = sp, control =
ctl_mixed)
  expect_s3_class(tr, "igsca_tree")
  expect_gt(partykit::width(partykit::node_party(tr)), 1L) # it actually split
  expect_identical(attr(tr, "igsca_info")$n_fail_full, 0L)
}

```

`expect_gt(width, 1)` is the minimum assertion that would have caught S1;
`expect_identical(attr(tr, "igsca_info")$n_fail_split, 0L)` is now the sharper one. Section 5 goes further.

4.4 T4. Only the partition selectors are expensive — mixed splitters are cheap

Revised from the first pass, which over-generalised the cost. The two paths differ by an order of magnitude, because only the partition **selectors** run permutation p-values; a mixed splitter re-scans candidates but inherits COIN variable selection from libcoin. Measured on this fixture:

Configuration	Wall clock
one 2-group MGA fit	0.585 s
mixed splitter, <code>maxdepth = 2</code> , <code>max_cuts = 8</code> , <code>R_test = 50</code>	1.5–2.6 s
partition selector, <code>maxdepth = 1</code> , <code>max_cuts = 3</code> , <code>R_test = 24</code>	14.0–15.3 s
partition selector, one node at <code>max_cuts = 20</code> , <code>R_test = 500</code>	~ 15 min
partition selector, one tree at <code>maxdepth = 3</code> (default)	~ 1.8 h

All 17 calls in the test file still use a bare `igsca_tree_control()`. The eight ctree-branch calls are fine at defaults; the nine partition-branch calls are not — at ~ 1.8 h each the file is on the order of half a day. Define two controls at the top and use them by branch:

```

ctl_mixed <- igsca_tree_control(R_test = 50L, maxdepth = 2L, max_cuts = 8L)
ctl_part  <- igsca_tree_control(R_test = 49L, max_cuts = 4L, maxdepth = 2L,
                               minbucket = 100L)

```

`R_test = 49L` matters only for `ctl_part`, and is worth a comment in the file. `permutation_pvalue()` returns $(1 + k) / (R + 1)$, so the smallest attainable p-value is $1/(R + 1)$, and the split criterion is a **strict** inequality (`crit > logmincriterion` $\Rightarrow$ `p < alpha`). At `R_test = 19L` the floor is exactly $1/20 = 0.05$, which does **not** clear `alpha = 0.05`: the partition family needs `R_test >= 20L` or it silently returns stumps for reasons that have nothing to do with the data. `49L` leaves comfortable headroom at $1/50 = 0.02$.

This does **not** apply to `ctl_mixed`. libcoin's Monte-Carlo p-value is a plain proportion with no add-one — measured, $0.74 = 37/50$ and $0.890 = 178/200$ exactly — so `p = 0` is attainable at any `R_test` and there is no such floor on the ctree branch. If a full-fidelity run matters, keep it behind `skip_on_ci()`.

4.5 T5. Nothing asserts the diagnostics the port exists to collect

Partly addressed. Section 5.4's three blocks now assert `n_split_scan` and `n_fail_split` on all three paths (dead kernel, working kernel, native). Still untouched: `n_fail_full`, `n_fail_node`, `n_fail_resample`, `root_criteria`, and the node-local scan cache in `argmax_split()` — the one piece of genuinely subtle logic in the port (`identical()` on closures as a cache key). See section 5.5.

5 Testing the mixed-splitter configurations

The mixed pairs (COIN variable selection + a model-comparison split point) are the configurations the suite is least able to check, because their failure mode is a **plausible-looking tree**, not an exception. Three measured facts shape what a useful test can look like.

(a) **The splitfun wiring works, but it is an undocumented contract.** `doTrees()` plants its kernel in `cc$splitfun` and relies on `partykit` reading it back out of the control object. A counting wrapper confirms it fires — 9 kernel calls on a `maxdepth = 2`, `max_cuts = 8` tree (1 at the root for `z_true`, 8 at node 2 for `noise_1`) on `partykit 1.2.29`. The template’s note says “verified against `partykit 1.2.28`”, so this contract has already survived one upgrade unverified. It deserves an explicit test.

(b) **The fixture cannot falsify cutpoint choice at the root.** `z_true` is a 2-level factor, so `candidate_partitions()` returns exactly **one** candidate:

```
length(candidate_partitions(1, dat$z_true, dat$z_true, max_cuts = 20L, minbucket =
30L))
#> 1
length(candidate_partitions(2, dat$noise_1, dat$noise_1, max_cuts = 20L, minbucket =
30L))
#> 20
```

Every kernel is forced to return the same split at the root. Any root-level assertion about **which** cutpoint a mixed splitter chose is vacuous, which is why no amount of rephrasing the existing `expect_no_error()` would have worked.

(c) **Kernels do disagree, on numeric covariates.** Pinning `whichvar` to `noise_1` and running `argmax_split()` directly:

```
FIT  cutpoint on noise_1 = -0.231765
DLi  cutpoint on noise_1 = -1.69654
DGi  cutpoint on noise_1 = -1.69654
```

So a differential assertion is meaningful — but **DLi and DGi legitimately agree**, both being model-implied indicator-VCV distances. Do not assert pairwise distinctness across all three.

A note on injection. Now that B2 has made `splitter` a `match.arg()` string, a test can no longer pass a custom kernel by value. Anything that needs to observe or subvert a kernel must go through `local_mocked_bindings()`, which rebinds the symbol in the package namespace where `doTrees()`’s `switch()` resolves it. Verified working on `testthat 3.3.2` (installed): the mocked counting wrapper below records the same 9 calls the by-value version did.

5.1 Kernel unit tests — the layer that was missing

Two MGA fits each, and the failure message names the broken kernel. This is the test that would have caught the missing `ndt_dists()` in seconds:

```
node_fixture <- function() {
  ind <- parseModel(model)$indicators
  ft <- try_fit(dat[, ind, drop = FALSE], model)
  list(model = list(object = ft$fit),      # what the trafo hands the kernel
        mf    = dat,
        ctrl  = list(collector = new_collector(), model = model,
                      indicators = ind, max_cuts = 20L, minbucket = 30L))
}
```

```

test_that("every split kernel returns a finite statistic", {
  fx <- node_fixture()
  gl <- fx$mf$noise_1 <= stats::median(fx$mf$noise_1)
  for (k in c("FITdiff", "DLi", "DGi")) {
    expect_true(
      is.finite(partition_stat(k, fx$model, fx$mf, seq_len(nrow(fx$mf)), gl, fx$ctrl)),
      info = k
    )
  }
})

```

`partition_stat()` converts a missing-function error into NA + `n_fail_resample++`, so `is.finite()` catches it. Observed values at the median split are `FITdiff = -0.00398`, `DLi = 0.0439`, `DGi = 0.0136` — note the negative FIT difference, which is exactly the case `partition_select()`'s `.Machine$double.eps` floor exists to handle, and worth a regression test of its own.

5.2 `argmax_split()` returns an admissible cutpoint

```

test_that("argmax_split returns an admissible cutpoint for every kernel", {
  fx <- node_fixture(); j <- match("noise_1", names(fx$mf))
  for (k in c("FIT", "DLi", "DGi")) {
    kern <- switch(k, FIT = split_max_fitdiff, DLi = split_max_dli, DGi =
split_max_dgi)
    sp <- argmax_split(kern, fx$ctrl$collector, fx$model, fx$mf,
      seq_len(nrow(fx$mf)), j, fx$ctrl)
    expect_s3_class(sp, "partysplit")
    br <- partykit::breaks_split(sp)
    expect_true(is.finite(br), info = k)
    expect_true(br > min(fx$mf$noise_1) && br < max(fx$mf$noise_1), info = k)
  }
})

test_that("different kernels place different cutpoints", {
  fx <- node_fixture(); j <- match("noise_1", names(fx$mf))
  cut <- function(kern) partykit::breaks_split(
    argmax_split(kern, fx$ctrl$collector, fx$model, fx$mf,
      seq_len(nrow(fx$mf)), j, fx$ctrl))
  # NB: DLi and DGi legitimately agree -- both are indicator-VCV distances.
  expect_false(isTRUE(all.equal(cut(split_max_fitdiff), cut(split_max_dli))))
})

```

These call the kernels directly rather than through `doTrees()`, so they are unaffected by B2 and need no mocking. This is where cutpoint choice **must** be tested, per fact (b) — at the tree level the root gives the kernel no freedom.

5.3 Assert that `partykit` actually calls the `splitfun`

```

test_that("doTrees installs its splitfun into partykit's split search", {
  calls <- new.env(parent = emptyenv()); calls$n <- 0L
  real <- split_max_fitdiff
  local_mocked_bindings(
    split_max_fitdiff = function(model, mf, subset, goes_left, ctrl) {
      calls$n <- calls$n + 1L
      real(model, mf, subset, goes_left, ctrl)
    }
  )
  set.seed(11)

```

```

tr <- doTrees(dat, model, covs, influence = "mat",
              splitter = "FIT", control = ctl_mixed)
expect_gt(calls$n, 0L) # the contract
expect_gt(partykit::width(partykit::node_party(tr)), 1L) # it split
})

```

Verified: `calls$n == 9, width == 3`. Capturing `real <- split_max_fitdiff` **before** the mock is what avoids infinite recursion. This test is what breaks loudly if a partykit upgrade stops honouring `ctrl$splitfun`.

5.4 Negative control — give the suite teeth

Status: implemented and passing (tests/testthat/test-postestimate_doTrees.R, 10 assertions across three blocks). One test alone would not have been enough: asserting only that a dead kernel **warns** leaves open whether a working kernel warns too, so the trio brackets the behaviour from both sides plus the kernel-free path.

```

ctl_mixed <- igsca_tree_control(R_test = 50L, maxdepth = 2L, max_cuts = 8L)

test_that("a dead split kernel is reported, not silently a stump", {
  local_mocked_bindings(
    split_max_dli = function(model, mf, subset, goes_left, ctrl) {
      stop("kernel is broken")
    }
  )
  set.seed(11)
  expect_warning(
    tr <- doTrees(data = dat, model = model, covariates = covs,
                  influence = "mat", splitter = "DLi", control = ctl_mixed),
    "produced no usable statistic"
  )
  info <- attr(tr, "igsca_info")
  expect_gt(info$n_fail_split, 0L)
  expect_identical(info$n_fail_split, info$n_split_scan)
  # The tree really is the stump the diagnostic is warning about
  expect_equal(partykit::width(partykit::node_party(tr)), 1)
})

test_that("a working split kernel records scans but no failures", {
  set.seed(11)
  tr <- expect_no_warning(doTrees(
    data = dat, model = model, covariates = covs,
    influence = "mat", splitter = "DLi", control = ctl_mixed))
  info <- attr(tr, "igsca_info")
  expect_gt(info$n_split_scan, 0L)
  expect_identical(info$n_fail_split, 0L)
  expect_gt(partykit::width(partykit::node_party(tr)), 1L)
})

test_that("the native split path never touches the kernel counters", {
  set.seed(11)
  tr <- doTrees(data = dat, model = model, covariates = covs,
                influence = "mat", splitter = "native", control = ctl_mixed)
  info <- attr(tr, "igsca_info")
  expect_identical(info$n_split_scan, 0L)
  expect_identical(info$n_fail_split, 0L)
})

```

Two things worth carrying into the rest of the suite. The reduced `ctl_mixed` keeps all three blocks to a few seconds — mixed splitters run no permutation test, so this costs nothing in fidelity (see T4). And `partykit::width()` returns a **double**, not an integer: `expect_identical(width(...), 1L)` fails on type alone. Use `expect_equal(..., 1)` or `expect_gt()`.

5.5 The scan cache

`argmax_split()`'s cache key is `identical()` on the splitter **closure** (R/helper_doTrees.R:294--296). A matched selector/splitter pair should reuse the selector's scan; a mismatched pair should rescan with its own kernel. Both are cheap to check at unit level with a pre-armed collector, and neither is covered today:

```
test_that("a matched pair reuses the scan and a mismatched pair rescans", {
  fx <- node_fixture(); j <- match("noise_1", names(fx$mf)); subset <-
  seq_len(nrow(fx$mf))
  coll <- fx$ctrl$collector
  sc <- scan_covariate("FITdiff", j, fx$model, fx$mf, subset, fx$ctrl)
  coll$scan <- stats::setNames(list(sc), as.character(j))
  coll$scan_subset <- subset
  coll$scan_splitter <- split_max_fitdiff

  expect_identical(                                     # matched -> cache hit
    argmax_split(split_max_fitdiff, coll, fx$model, fx$mf, subset, j, fx$ctrl),
    sc$split)
  expect_false(isTRUE(all.equal(                         # mismatched -> rescan
    argmax_split(split_max_dli, coll, fx$model, fx$mf, subset, j, fx$ctrl),
    sc$split)))
})
```

Note this test must **not** be combined with `local_mocked_bindings()` on the same kernels: the cache key is closure identity, and a mock would change it.

5.6 What none of this proves

The five layers above establish that the mixed splitters **run, are invoked, and discriminate**. They do not establish that a mixed splitter finds the **right** cutpoint, because this fixture has no numeric covariate with a true threshold — `z_true` is binary and `noise_*` are noise. A recovery test needs a fixture where a numeric covariate drives a known group difference at a known cutpoint, then asserts the recovered breakpoint falls within a tolerance of it. That is the only test that checks the mixed configurations are statistically correct rather than merely alive, and it is worth building if the mixed pairs are a real study arm rather than a convenience.

6 Packaging and API

6.1 P1. `document()` has been run — now prune the export list

Status: inverted. `NAMESPACE` has been regenerated and `man/` populated, so the first pass’s complaint is resolved: `igsca_tree_control` and the `split_max_*` kernels are now reachable, and `cSEM::doTrees(control = cSEM::igsca_tree_control(...))` works.

The problem is now the opposite one. 26 new symbols are exported, and most are internal machinery with no standalone meaning:

```
argmax_split csem_converged fit_csem grow_extree idx_permutation
ndt_dists ndt_pools node_group_data partition_select partition_stat
permutation_pvalue scan_covariate select_ndt_dgi select_ndt_dli
select_npt split_max_dgi split_max_dli split_max_fitdiff try_fit
candidate_partitions influence_mat influence_vec ...
```

Every one of these is now public API you are committing to keep stable, and each carries a `man/*.Rd` that R CMD check will scrutinise (see P3). Replace `@export` with `@keywords internal` or `@noRd` on all of them. B2’s string-valued `splitter` has already removed the last **user-facing** reason to expose the `split_max_*` kernels — but keep them (or their `partition_stat()` backend) reachable from the test file, since the unit tests in section 5 call them directly. A defensible public surface is `doTrees`, `igsca_tree_control`, `bdiagFit`, `root_criteria`, and the two S3 methods.

6.2 P2. `:::` and `::` into the package’s own namespace

Location: `R/helper_doTrees.R:572--573` and `R/helper_doTrees.R:608`

Unchanged:

```
Sc = cSEM:::bdiagFit(single_fit, .n_blocks = 2L, .type_vcv = "construct"),
Si = cSEM:::bdiagFit(single_fit, .n_blocks = 2L, .type_vcv = "indicator")
...
cSEM::calculateFIT(mga$fit) - cSEM::calculateFIT(model$object)
```

Both are defined in this package (`bdiagFit` at `R/helper_doTrees.R:26`, `calculateFIT` at `R/helper_assess.R:2013`). R CMD check flags the `:::` form (“a package almost never needs to use `:::` for its own objects”), and both break under `load_all()` if the installed and sourced versions diverge.

The new code makes the inconsistency starker rather than better: `ndt_dists()` (`R/helper_doTrees.R:960--966`) calls `bdiagFit()`, `calculateDG()` and `calculateDL()` bare and correctly, sitting 390 lines below `ndt_pools()` doing the same thing through `cSEM:::`. Drop the prefixes.

6.3 P3. Roxygen placeholders — now materialised as `.Rd` files

`document()` has turned the placeholder blocks into committed documentation. `man/idx_permutation.Rd` in full:

```
\title{Title}
\usage{idx_permutation(n, R, strata = NULL)}
\arguments{
  \item{strata}{}
}
\description{
  Permutation resampling: sample WITHOUT replacement, within each stratum, ...
}
```

Three R CMD check problems in nine lines: the title is literally `Title`, `n` and `R` are undocumented while `strata` has an empty description, and there is no `\value{}`. That pattern repeats across the 20-odd new `.Rd` files. P1's pruning is the cheapest fix — `@noRd` deletes the file and the warnings with it — and whatever survives as public API needs a real title, real `@param` text and a real `@return`.

Two specific errors in `man/doTrees.Rd`:

- **\value is wrong.** It claims class `modelparty` and `party`. Actual class is `c("igsca_tree", "constparty", "party")` on both paths; there is no `modelparty` anywhere in the package.
- **Half the arguments are undocumented.** `data`, `model` and `covariates` did inherit from `csem_arguments` (roxygen matched them to the dot-prefixed `.data` / `.model` / `.covariates` entries). `influence`, `splitter` and `control` did not — they appear nowhere in `\arguments{}`, which is an R CMD check WARNING. They are also the three arguments a reader most needs explained, since `influence` selects between two entirely different algorithms and `splitter` is constrained by which one you picked (B2's `stopifnot`).

6.4 P4. Argument naming departs from the package convention

Every other exported cSEM entry point takes dot-prefixed arguments (`.data`, `.model`, `.approach_weights`, ...). `doTrees()` takes bare `data`, `model`, `covariates`, `influence`, `splitter`, `control`.

(The first pass claimed this also broke `@inheritParams`; that was wrong — roxygen matched the dotted entries to the undotted parameters, as P3 shows. The case for renaming rests on API consistency alone, which is still a good enough reason while the branch is young. Doing it together with B2's now-settled signature would keep the churn to one commit.)

6.5 P5. `fit_csem()` hard-codes the estimator with no override

Location: `R/helper_doTrees.R:152--164`

```
fit_csem <- function(.data, .model, .id = NULL) {
  csem(.data = .data, .model = .model, .id = .id,
    .approach_weights = "GSCA", .disattenuate = TRUE,
    .conv_criterion = "sum_diff_absolute", .iter_max = 100,
    .GSCA_modes = "CCMP", .tolerance = 0.0001)
}
```

Reasonable as a study default, but `doTrees()` gives the user no way past it — no ..., no argument. Since the tree machinery is estimator-agnostic (nothing outside `partition_stat()`'s FIT branch is GSCA-specific), threading ... from `doTrees()` through `try_fit()` into `csem()` costs almost nothing and makes `doTrees()` usable for PLS-PM trees too. At minimum, document that the estimator is fixed — especially now that `fit_csem` is exported.

6.6 P6. No input validation

`doTrees()` validates `influence` and `splitter` (via `match.arg`) and nothing else. Worth adding, since all three cases currently produce errors from deep inside `partykit` or `csem()`:

```
stopifnot(
  "`data` must be a data.frame" = is.data.frame(data),
  "`covariates` must be columns of `data`" =
    length(covariates) > 0L && all(covariates %in% names(data))
)
missing_ind <- setdiff(indicators, names(data))
if (length(missing_ind)) {
  stop2("The following indicators are not columns of `data`: ",
```

```
    paste(missing_ind, collapse = ", ")
}
```

(`stop2()` is the package's own error helper, `R/zz_utils.R:114`.)

7 What the port got right

Worth recording, since these are deliberate improvements over the template rather than accidents:

- **Indicators from the model, not the data.** `parseModel(model)$indicators` (R/`postestimate_doTrees.R:33`) replaces the template’s `setdiff(names(data), covariates)`. The template silently treated any stray column as an indicator; the port does not, and now tolerates extra columns in `data`. The two orderings differ (`x31, x32, ..., x11` vs `x11, x12, ...`), which is why the recorded snapshot’s model formula reads `~x31 + x32 + x33 + x11 + ...` — expected, not a bug.
- **The unified dispatch is the right cut.** Branching on `influence` rather than exposing two constructors correctly reflects that the COIN and partition families differ only in how the node statistic is produced.
- **`idx_permutation()` and `ndt_dists()` were ported carefully.** The $R \times n$ orientation and the (`Sc_pool`, `Si_pool`, `mga_fit`) argument order both match their call sites, and `ndt_dists()` uses unprefixed internal calls — the convention the older `ndt_pools()` still violates.
- **B2 was fixed properly rather than patched.** Making `splitter` a `match.arg()` string closed the `logical(0)` hole structurally instead of special-casing `NULL`, and kept the one `stopifnot()` that still carries meaning. Both guard paths now produce errors a user can act on.
- **`influence_ssr` → `influence_vec`** reads better next to `influence_mat`. The stale “COIN_ssr” references in the comments (R/`helper_doTrees.R:113, :514`) should follow the rename, as should the now-incorrect provenance comment at `:535`.
- The scan cache, minbucket filtering, and `.Machine$double.eps` flooring all survived the port intact and behave correctly under test.

8 Suggested order of work

#	Why in this position
1	The constant already exists; the other 17 calls still use bare defaults — promote §5.4’s <code>ctl_mixed</code> to the top of the file, add <code>ctl_part</code> at <code>R_test</code> <code>>=</code> <code>20L</code>
2	The signature has settled, so snapshots are now safe to regenerate; T2 fails outright once T3 lands — regenerate snapshots, drop

#	Why in this position
	length()==5, split expect_no_error()
3	§5.1 Cheap, fast, and they pin the contracts the port depends on 5.3 — kernel, argmax_split, and splitfun- wiring tests
4	§5.1 Subtlest logic in the port; the only collector field still unasserted + §5.5 — scan- cache tests
5	§5.1 Cleanup; P4's rename pairs naturally with B2's settled signature. Note <code>attach_leaf_fits()</code> , <code>Drop_inner_node_objects()</code> and <code>warn_dead_splitter()</code> are already @noRd — prune exports, drop :::, fix \value, validate inputs
6	§5.1 The counter is new and nothing tests it; the default 30L admits unfittable leaves follow- up — assert n_fail_leaf and raise minbucket for large models
7	§5.6 Optional; only if the mixed pairs are a real study arm — numeric- cutpoint fixture

#	Why in this position
8	\$cc\$ — \$cc\$ optional: replace the cc\$bonferroni override with a length-2 testtype